Vision for Autonomous Robots

Lab Project 3: Mapping & Route Planning

Lisann Becker (12558141)
Batu Helvacioğlu (14992760)
Marina Orozco González (15214052)

1 Introduction

In this lab, we explored Structure from Motion (SfM) to reconstruct a 3D representation of a maze environment using video frames captured by the RAE robot. The maze, with images displayed on its walls, served as setting for navigating through and mapping the environment.

Using VisualSfM, we process video frames to detect keypoints, match them across frames, and generate a sparse reconstruction, which is then refined into a dense 3D representation. This reconstruction provides key insights into the environment, including detailed geometry and features of the maze.

Since our primary objective is to reconstruct only the maze and not the entire room, we perform post-processing on the dense reconstruction to remove features unrelated to the maze. We investigate the conditions that can automate this cleaning process by identifying common patterns in feature extractions and analyzing which aspects depend on the software's specifics.

Finally, we implemented a 2D representation of the maze by projecting it onto a 2D occupancy grid. This bird's-eye view of the maze enabled us to design a route-planning algorithm for navigating through and escaping the maze. For this, we utilized a basic version of Dijkstra's Algorithm and enhanced it with a penalty map to optimise the process.

2 Session 1: Structure-from-Motion

2.1 Pre-process the video into a sequence of images, and do off-line SfM.

Firstly, we need to convert the provided video into frames. For this, we implemented a custom script that extracts the video frames into a designated folder. To improve reconstruction quality, we decided to utilise every single original frame for SfM. On the one hand, computational cost of SfM scales with the number of frames, as each additional frame increases the number of keypoints to identify and match across frames. However, using fewer frames reduces quality of the final reconstructed point clouds, as matching them will be harder: if you try to match the keypoints of two non-consecutive frames which hardly show the same objects, the 3D reconstruction of the scene can fail. Therefore, we preferred to run the computation for longer rather than sacrificing quality of the maze reconstruction.



Figure 1: Matching of keypoints between two frames.

After frame extraction, we used the Scale-Invariant Feature Transform (SIFT) algorithm to detect keypoints in each frame. Then we matched the keypoints of each frame to its preceding frames' keypoints (full match) to establish correspondences between frames 1. Sparse reconstruction followed, which uses the matched keypoints to estimate camera poses and generate an initial sparse 3D point cloud which provides the basic geometry of the scene. We applied bundle adjustment to

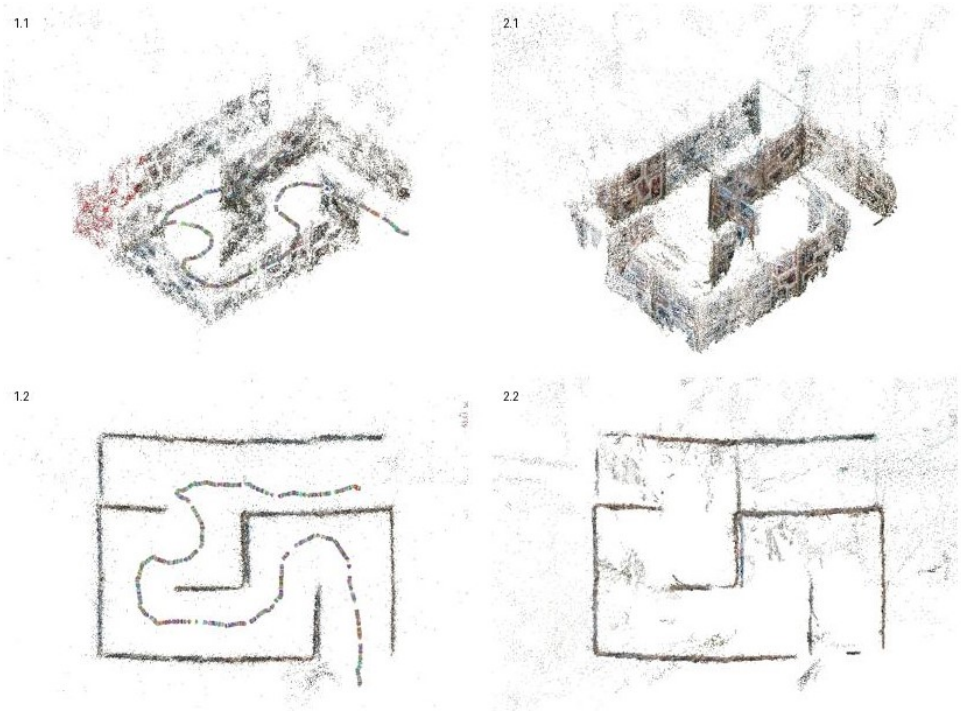


Figure 2: Raw sparse (1) and dense (2) reconstruction results of Maze 1, shown from the side and from bird's-eye view.

optimise camera parameters and 3D point positions, ensuring that our point cloud was as consistent as possible.

Lastly, we ran dense reconstruction to refine the sparse model, producing a detailed 3D point cloud - a comparison between sparse and dense reconstruction results can be found in figure 2.

2.2 Discuss and analyze the weakness. (5 pts)

The most notable flaw in the reconstruction results for Maze 1 was that sparse regions were present even after dense reconstruction. For example, some corners of the maze were not reconstructed properly, as shown in 3. We attribute these issues to two factors: the relatively low resolution of the video frames (640 x 400) and insufficient frame coverage in certain areas. For example, one corner near the end of the maze appeared only in 14 out of 683 frames. This makes it difficult for the SfM pipeline to extract enough keypoints and correspondences to reconstruct the regions accurately.

These weaknesses have important consequences for our downstream tasks: path-finding algorithms like Dijkstra's could underperform due to false gaps in the walls and for instance, attempt to lead the robot through non-existent shortcuts.

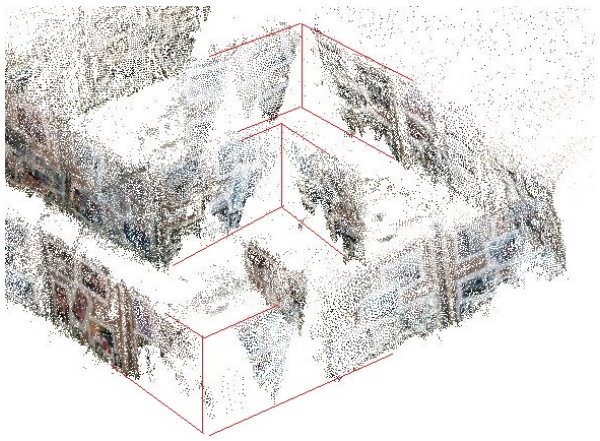


Figure 3: Example of missing corners in the reconstruction of Maze 1 - red lines show the geometry of the real maze which has not been successfully reconstructed due to low frame resolution and insufficient frame coverage.

2.3 Figure out a way to improve the reconstruction results. (5-pts)

To avoid similar issues for Maze 2, we increased the video resolution and number of frames for Maze 2, which lead to much higher higher reconstruction

quality - this is shown in Figure 4 and discussed in more detail in section 2.4. Higher-resolution frames improve keypoint detection, and more frames provide better coverage of all areas including corners.

We also used dilation and erosion on the 2D occupancy grid of Maze 1, to close gaps in walls in sparsely reconstructed areas which is shown in 10. However, this complicated distinguishing poorly covered but relevant areas, like corners, from incorrectly detected keypoints, exaggerating some illusory blobs of keypoints in the maze reconstruction.

These improvements, combining higher frame rate and resolution and post-processing, significantly enhance reconstruction accuracy.

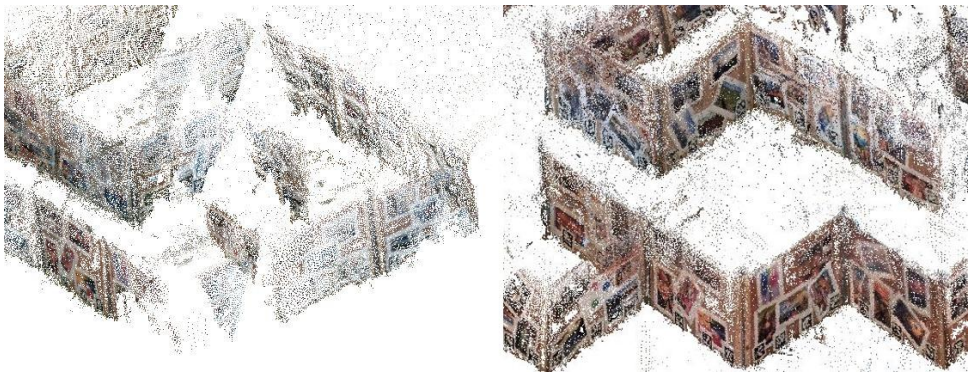


Figure 4: Because it utilises fewer keypoints, the dense reconstruction of Maze 1 (left) lacks important detail like maze corners. Despite being larger, such detail is captured well in the reconstruction of Maze 2 (right) as it is reconstructed from five times as many frames and almost 12 times as many keypoints.

Part-2: On New Maze in the Second Week

2.4 Find the Topic List for Video Recording. (5-pts)

To collect video frames of the maze, we created a custom node called the image_capture_node. This node supports a selection of cameras and multiple functionalities: manual photo capture upon key press, periodic photo capture (1 frame per second), and video recording. The latter was the relevant recording function for collecting a video of the maze from the RAE robot and was run with the command: `ros2 run image_capture image_capture --video y`. It is important to notice that `q` must be pressed to stop so that the video is saved.

The video function of the node saves individual frames from the front-right camera of the RAE robot (`rae\right\image_raw`) while also compiling them into an MP4 video. It is crucial to press q to stop the recording and finalise saving the video file properly. The node stores all frames captured during one session in a dedicated folder and timestamps them which simplified excluding unnecessary beginning and end frames.

The recorded video for Maze 2 had a total of 2,784 frames, significantly more than the 682 frames captured for Maze 1 in the first session. This increase resulted in a point cloud with 1,354,969 points for Maze 2 compared to 117,207 points for Maze 1 and a significant improvement in reconstruction detail 4. However, this improvement increased computation time for keypoint extraction and matching, as the complexity of key point matching grows quadratically with the number of frames.

2.5 Collect and Pre-processing of the Collected Video. (5-pts)

As explained in the previous section, we implemented a script to automatically store the recording video as frames. To check the accuracy of the `image_capture_node`'s video function, we wrote a supplementary script to split a video into individual frames. The frames matched those captured by the node, confirming it worked reliably.

The `image_capture_node` saves each frame as an image while also creating an MP4 video in real-time. This dual output takes away the need for manual frame splitting, simplifying pre-processing significantly. For the following SfM tasks, we could directly utilise the extracted frames.



Figure 5: Some of the frames captured by the robot that are used to reconstruct the second maze.

Some of the images we captured are visualised in Figure 5.

2.6 Do SfM on the new maze.

The second maze was slightly larger than Maze 1 and contained an open middle structure so that multiple paths could lead the robot to its goal, compared to only one possible route in maze 1. Taking into account that the reconstruction of Maze 1 was limited by low frame coverage of some maze regions and relatively low image resolution, we firstly utilised over four times as many frames. As described above, this resulted in better reconstruction quality. The improvement was further enhanced by choosing a higher resolution of the video frames, at 906 x 460 pixels compared to 640 x 400 for Maze 1 which allowed for more accurate keypoint detection and matching across frames.

We followed the same SfM pipeline detailed in section 2.1, but to reduce computation time, we cropped out the ceiling portion of the frames, which was irrelevant for the maze reconstruction.

The significant increase in the number of comparisons explains the longer computation time for the second maze. We applied full matching again, comparing each frame to its preceding frames. For Maze 1 with 682 frames, the total number of comparisons is given by $\frac{N(N+1)}{2}$, resulting in $\frac{682 \cdot 683}{2} = 232,903$ comparisons. Similarly, for Maze 2 with $N = 2784$ frames, the total number of comparisons is $\frac{2784 \cdot 2785}{2} = 3,876,720$. This corresponded to a computation time increase from 2.1 hours for Maze 1 to 33.2 hours for Maze 2. The reconstruction results, including both sparse and dense reconstructions, are shown in Figure 6.

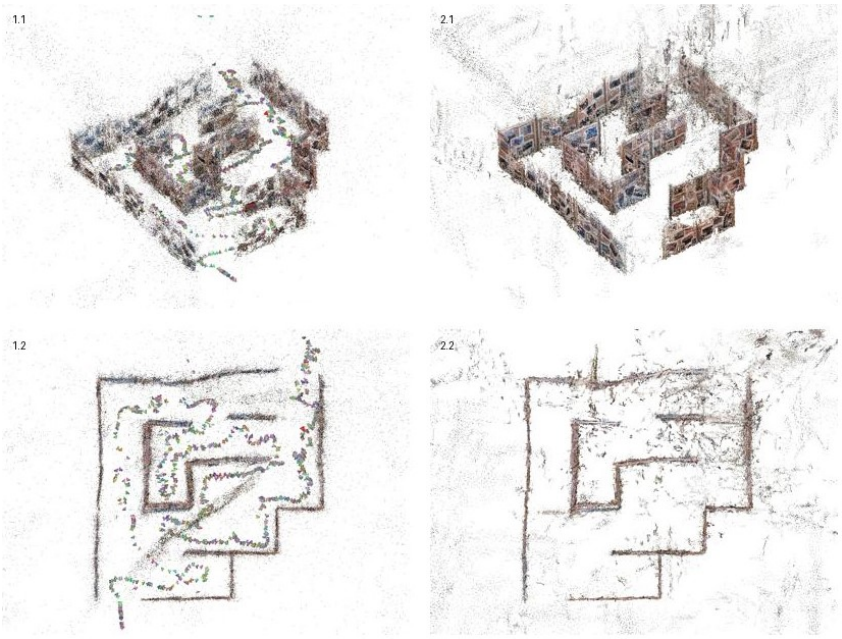


Figure 6: Raw sparse (1) and dense (2) reconstruction results of maze 2, shown from the side and from bird's-eye view.

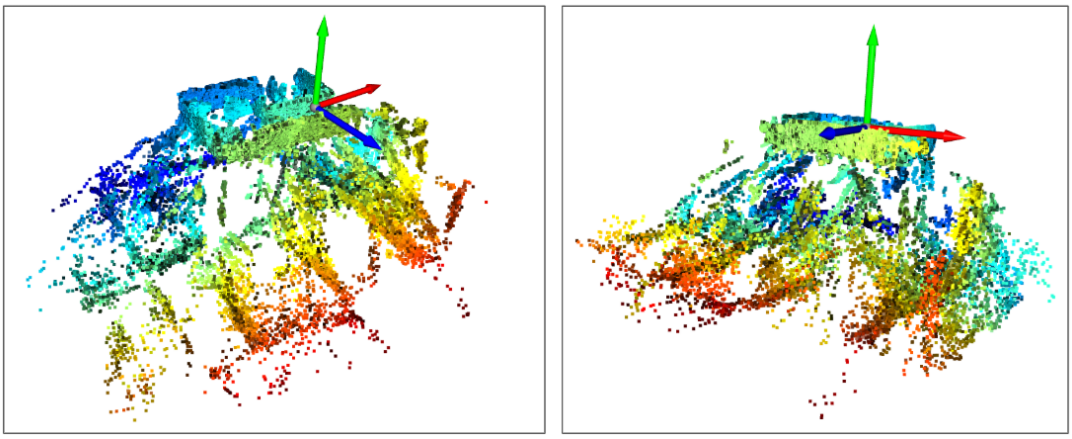


Figure 7: Keypoints of the maze visualized by `open3d`. We can visualize the axis of the reference system that the maze was computed on. In this coordinate system, the red, green and blue arrows correspond to X, Y and Z axis respectively. The location respect to the lab is such that the Y (green) axis points to the floor.

3 Session 2: Mapping

3.1 Remove the Keypoints away from the Maze. (10-pts)

We receive the dense reconstruction of the maze as a `.ply` file. While keypoint filtering can be done manually in visual editors like `MeshLab`, our goal is to automate this process in Python.

Using the `open3d` library, we can import and visualise the keypoints, as shown in Figure 7. Figure 7 also clearly shows the need to filter the large amount of keypoints outside of the maze which would cause difficulties topological mapping and route planning downstream.

To filter out keypoints far from the maze, we remove all points with a Z-coordinate less than -2 or greater than 0, based on experimental observations. The "greater than 0" condition holds for all mazes, as the robot's upside-down camera position ensures the Z-axis points from the ceiling to the floor, with the origin at the floor. However, the minimum Z value must be recalibrated for each maze, as it varies with

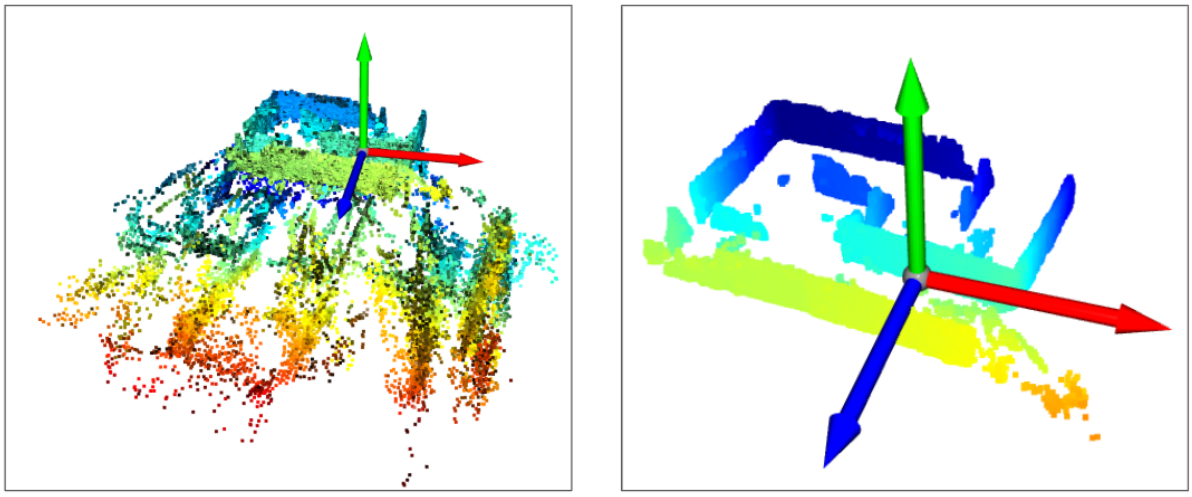


Figure 8: Before (left) and after (right) applying the automatic filtering out of keypoints outside of the maze.

the keypoint distribution. We can make no assumptions about the X-Y coordinates, as they depend on the robot's starting position. The results of this filtering are shown in Figure 8.

The Z-threshold filtering is highly effective, removing most of the irrelevant points above the maze. Z-filtering alone is naturally effective in our context, as the maze walls block the robot's view of the X-Y plane, leaving only points above the walls, like the ceiling, visible. However, some keypoints within or along the walls of the maze remain.

Next, to further refine the point cloud, we use statistical outlier filtering which analyses the local neighbourhood of each keypoint to identify and remove points that are isolated from their neighbours. We used the `remove_statistical_outlier` function, considering 10 nearest neighbours and a standard deviation ratio of 2. Keypoints that exceed the threshold—defined as the mean distance plus the standard deviation ratio times the standard deviation—are classified as outliers and removed.

By combining z-threshold filtering with statistical outlier removal, we produce a much cleaner point cloud representation 8, and proceed in building a higher-quality topology map.

3.2 Point Cloud to Topology Mapping. (10-pts)

1. As mentioned before, we can assume that the maze's walls are vertical (Z axis in the 3D keypoints representation). This allows us to project all keypoints onto the X-Y plane, hence, constructing a bird's-eye view of the maze without losing information. We grid the view to enable applying path-finding algorithms like Dijkstra and define cells including more than 10 keypoints as occupied. This 2D occupancy grid of the maze is shown in image (2) of Figure 9.
2. Next, we solve that the maze will appear rotated in different angles within the X-Y plane. Our goal is to determine the rotation angle we need to align the maze's lines with the X-Y axes. We follow these steps:
   (a) We perform a Hough transformation from `skimage` to detect straight lines.
   (b) With `hough_line_peaks`, we keep only the most prominent results. Express the angles in degrees, and normalise them to the range 0–180º.
   (c) Create a histogram with 180 bins to group the angles and identify the most prominent one, corresponding to either the maze's most vertical or horizontal line.
   (d) Lastly, calculate the difference between this angle and 90º to determine the rotation needed. Alternatively, we could use the difference from 180º, as rotating the maze by 90º is sufficient for alignment.

   The resulting map can be seen in Image (3) of Figure 9.

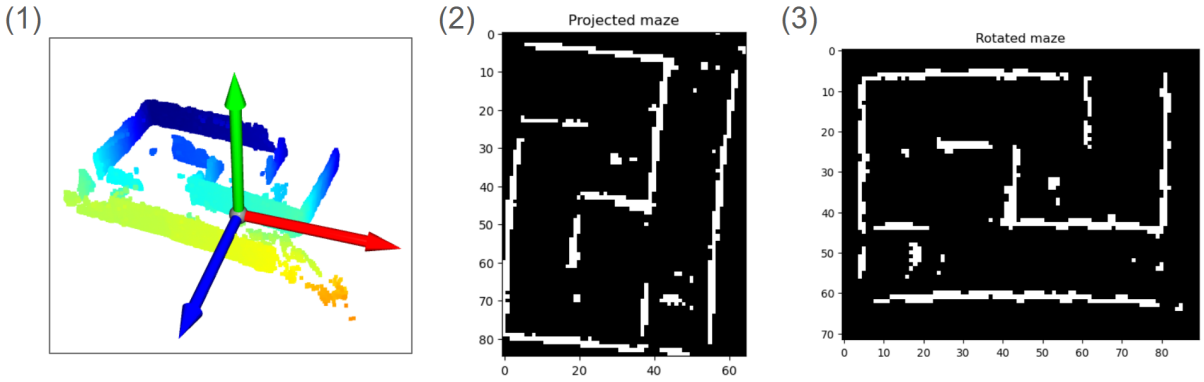


Figure 9: 3D keypoints representation of the maze (1), its 2D projection to create a bird's-eye occupancy grid of it (2), and the resulting rotated map after applying automatic rotation with Hough transformation (3).

3. The reason for rotating the map is for the following: until now, the projected map appears with many unconnected parts of the wall, which is the issue described in 2.3 and 2.4. However, we need to connect them to obtain the real maze's map. For this, we apply erosion and dilation with vertical and horizontal kernels, once again making use of the perpendicular properties of the walls of the mazes. For those, we used `cv2.erode` and `cv2.dilate`. Specifically:

   (a) Remove of isolated segments, defined as those containing only 1, 2 or 3 pixels. We made this making use of `cv2.CC_STAT_ARE` function.

   (b) Vertical dilation with a 8×1 kernel size.

   (c) Vertical erosion with a 4×1 kernel size.

   (d) Horizontal dilation with a 1×6 kernel size.

   (e) Horizontal erosion with a 1×5 kernel size.

   These steps are visualised in Figure 10. The specific kernel sizes were chosen through empirical testing on the Maze 1. While this configuration should work on other mazes, better results would have been obtained if we had adjusted it dynamically to each maze's properties and each resulting map from previous steps.

3.3 Display Your Map. (5-pts)

Our final map can be seen in Figure 11. We followed same steps for Maze 2 with results portrayed in 12. For the second maze, we changed the Z-threshold to filter keypoints to -30 and the resolution of the 2D occupancy grid. These changes were necessary as the higher number of keypoints of Maze 2 led to a much bigger scale of the point cloud coordinates output by VisualSfM.

3.4 Discussion and Future Work. (5-pts)

Figures 11 and 12 demonstrate that our results for both mazes are satisfactory and achieved an approximate representation of the maze. However, some flaws are evident for Maze 1 whose keypoints were too sparse at times, leading to gaps in walls that should be continuous. As discussed in Sections 2.2, 2.3, and 2.6, the abundance of frames and higher resolution in the Maze 2 recording resulted in a denser reconstruction, preventing similar issues.

Without dilation, wall gaps could mislead the robot into perceiving them as escape paths, causing collisions. A possible solution to address this would be implementing online fine-tuning, dynamically refining the maze representation as new frames are processed and additional keypoints are added. However, dilating the walls is a practical solution in our context, as it provides padding that accounts for the robot's thickness.

Some deviations from the real maze are expected, as features and surfaces are estimated solely from the video. For example, walls visible from both sides may be interpreted as two distinct surfaces but later

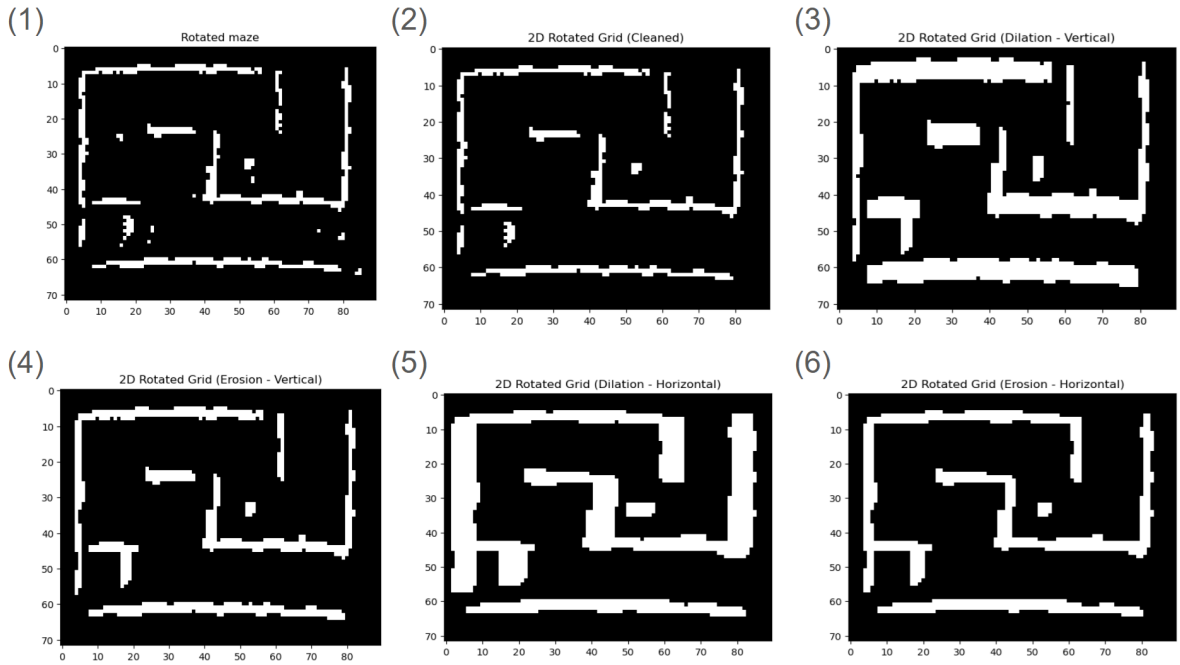


Figure 10: Dilation and erosion steps applied to close the walls from our 2D maze representation.

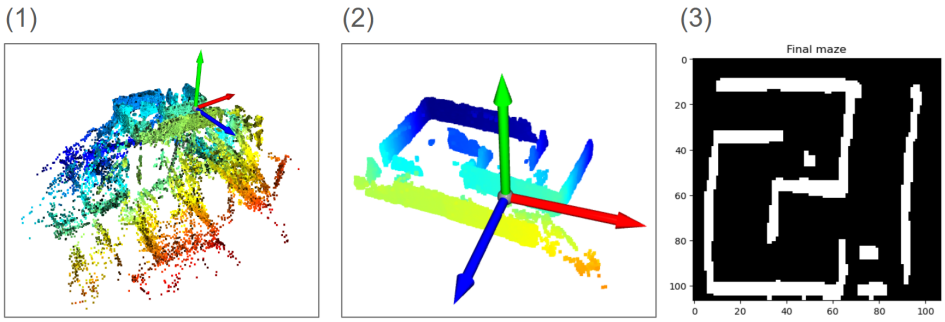


Figure 11: 3D keypoints of the first maze, filtered keypoints and final maze.

combined into a single wall during post-processing, overly increasing wall width. A potential improvement could be to impose the known wall width, enhancing the accuracy of the estimated distances between walls and better reflecting the navigable space for the robot.

4 Session 3: Route Planning

4.1 Route Planning by Dijkstra's Algorithm (15-pts)

As a post-processing step before Dijkstra we added a border to the grid to ensure that the path found by the algorithm does not circumvent the maze. This is done automatically by finding coordinates of the topmost, bottommost, leftmost and rightmost obstacles in the grid. The cells outside of these extreme points are marked as obstacles.
We implemented Dijkstra to find the shortest path that avoids the walls. Our implementation works by storing potential next cells and the distances to those cells from the start location in a heap.

1. We initialize all cell distances as infinity except the start location. The start location is initialized with distance 0 and added to the heap.
2. The cell with lowest distance is popped from the heap.
3. If the cell is an obstacle it is skipped.

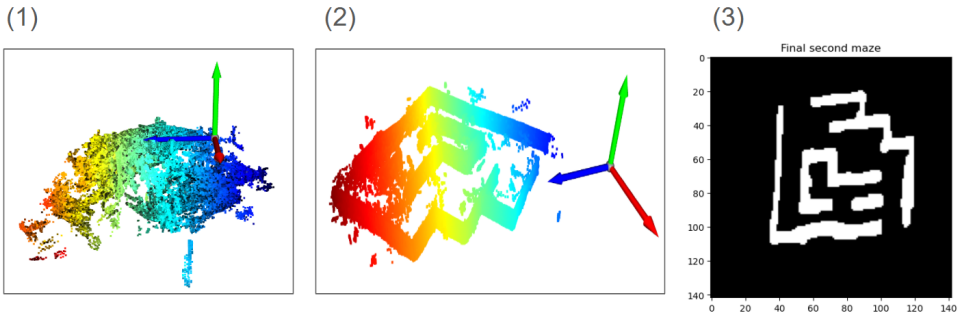


Figure 12: 3D keypoints of the second maze, filtered keypoints and final maze.

4. Neighbors of the cell are added to heap if they satisfy the following conditions:
   (a) The cell is inside the grid.
   (b) The cell is not an obstacle.
   (c) The new distance to the cell is (current distance + 1) is less than the old distance that is stored in an array.
5. Save the current node as the previous node of the neighbor cells that satisfy the requirements.
6. Repeat steps 2-4 until the goal is reached or the heap is empty.
7. Backtrack using the previous nodes list to get the path.

4.2 Discuss and analyze the planned route (5-pts)

Our planned routes for Maze 1 and 2 can be seen in Figures 13 and 14, respectively. Both paths are the shortest but are impractical without dilation, as they run too close to walls, risking collisions. Representing the robot as a single point in the grid overlooks its size, leading to real-world issues where the robot is likely to hit a wall. Another weakness of Dijkstra is that a diagonal path would be shorter than two straight paths joined by a 90 degree turn real life. This is a general issue using a grid representation and Manhattan distance.

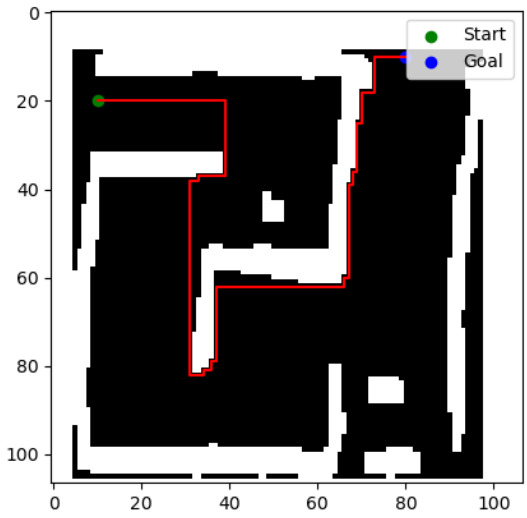


Figure 13: Planned route of maze from week 1 using Dijkstra.

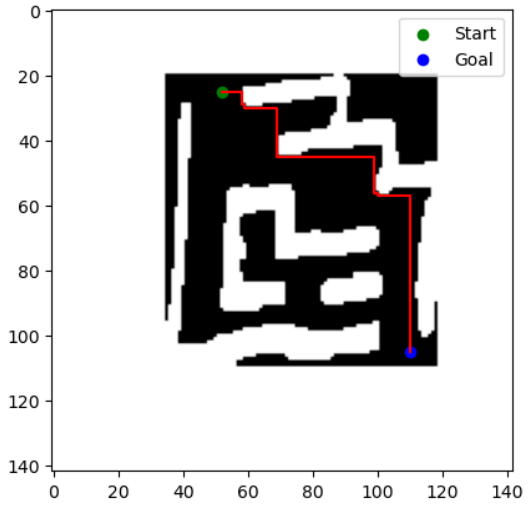


Figure 14: Planned route of maze from week 2 using Dijkstra.

4.3 Route Planning with Your Improved Version (10-pts)

We adapted Dijkstra's algorithm by incorporating a penalty map to prevent paths from being too close to the walls. We compute the penalty map as follows:

1. Invert the grid.
2. Use Scipy distance_transform_cdt to get Manhattan distance to obstacles for each cell in the grid.
3. Normalize the distance map, and scale by max penalty parameter 10 to obtain penalty map.

We then add the penalty of each cell to the distance in Step 4c of Dijkstra's Algorithm. Resulting paths seen in Figure 15 and 16 are far away from the walls of the maze. This is a much more practical path to follow in real life. We managed to avoid most walls in the maze except for the first corner in maze 1. This is due to an hallucinated obstacle in the maze. We get increased penalties around the hallucinated obstacle and therefore failing to avoid the first wall as it can be seen from the visualised penalty map in Figure 17.

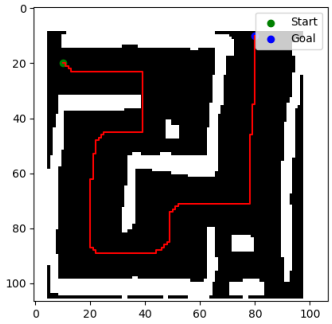


Figure 15: Planned route of maze from week 1 using Dijkstra with penalty map.

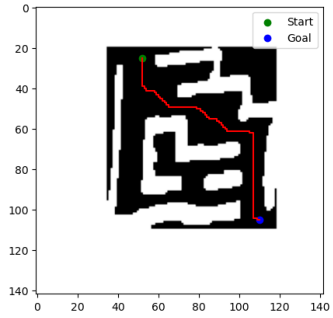


Figure 16: Planned route of maze from week 2 using Dijkstra with penalty map.

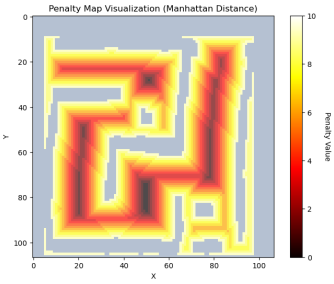


Figure 17: Penalty map of maze 1.

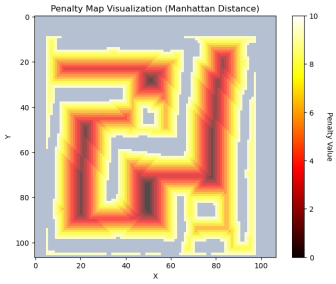


Figure 18: Penalty map of maze 2.

5 Conclusion

In this final lab, we explored Structure from Motion (SfM) to generate 3D reconstructions from video frames, successfully applying it to a real maze containing images on its walls. Using VisualSfM, we identified its potential and limitations, balancing computational demands with reconstruction quality. Despite sparse areas in Maze 1, denser reconstruction in Maze 2 demonstrated the benefits of higher resolution and frame coverage, capturing details like wall images and parts of the room.

We processed the 3D reconstruction to isolate the maze, applying filtering techniques and creating a bird's-eye view occupancy grid projection. While the initial maps showed gaps in walls due to sparse keypoints, post-processing steps like dilation resolved these issues, allowing us to produce refined 2D maps. These maps enabled route planning using Dijkstra's algorithm, which we further improved using a penalty map that ensured paths avoided walls, creating safer navigation routes for the robot.

Overall, we addressed the key challenges of the lab and gained valuable insights into the complexities of SfM, point cloud processing, and route planning.